PORTFOLIO PROJECT 03 · OBSERVABILITY & DEVOPS

endpoint-monitoring-stack

One Grafana dashboard and alert rules for hundreds of application servers – version-independent, provisioned from code, deployed with one command.

E2E ✓ verified run

Docker Compose · Prometheus · blackbox_exporter · Grafana · PromQL · Python · GitHub Actions

Sallehuddin Mohd Zakinuddin · Senior Application Developer · Portfolio project on GitHub

1 Problem Statement

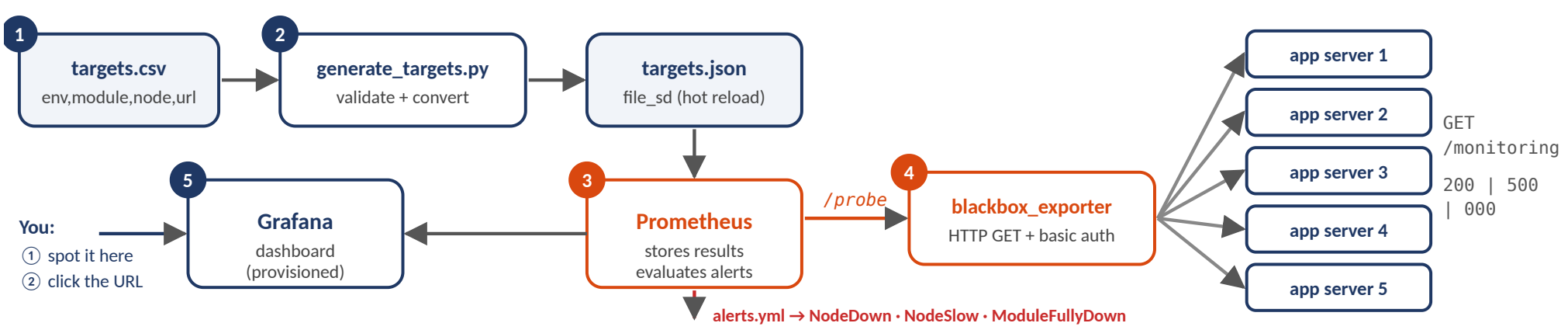
A multi-module Java application runs on **hundreds of application servers** across several environments. Each server has its own JavaMelody `/monitoring` page, and checking health meant **opening the URLs one by one**.

- No single view of "what is down right now?"
- No early warning – slow nodes were only noticed after users complained.
- The URL list lived in a spreadsheet that drifted from reality.

2 Objectives

1. **One dashboard** showing every node: up/down and response time.
2. Workflow: **spot the failing node in Grafana → click through** to its own JavaMelody page to dig in.
3. **Alerts** for down, slow, and "whole module down".
4. Inventory kept as a simple CSV; everything else **generated and auto-provisioned**.
5. One command to deploy anywhere: `docker compose up`.

3 System Architecture



① **Inventory.** One row per node. The only file a person edits.

② **Generate.** Rejects duplicate / malformed URLs, writes Prometheus targets with env, module, node labels.

③ **Prometheus** asks blackbox to probe each URL every 30 s, stores results, evaluates alert rules.

④ **blackbox_exporter** does the real HTTP GET with login – exactly what a person's browser would do.

⑤ **Grafana** dashboard and data source are provisioned from files – no manual clicking.

4 Key Design Decision – why blackbox?

Option A – native scrape

`GET /monitoring?format=prometheus`

✓ rich metrics: JVM heap, SQL time, hits

✗ **only exists in JavaMelody 1.73+**

✗ our servers ran an older version

→ HTTP 500 on every node

✗ upgrading every app just for monitoring = not realistic

Option B – blackbox probe ★

`GET /monitoring (same URL as a human)`

✓ **works with ANY app version**

✓ zero change on the servers

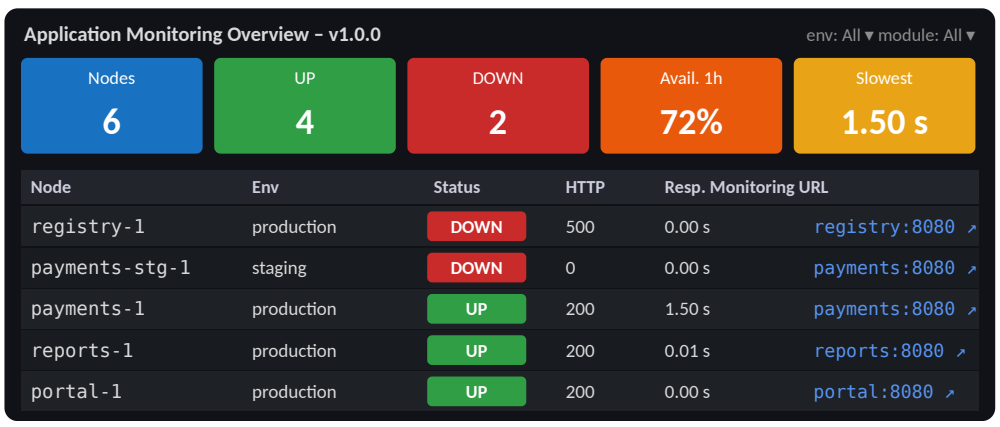
✓ up/down + response time per node

✗ no JVM internals in Grafana

→ acceptable: deep-dive still happens on the node's own page

Lesson: check the **version** of what you are integrating with before designing around a feature. I confirmed it from a node's own log, and Prometheus' *Status → Targets* page showed the real error (500), not a Grafana bug.

5 The Dashboard



Failures sort to the top; every monitoring URL is a clickable link. Count tiles use `… OR vector(0)` so they show **0** instead of "No data" when nothing is down – a bug I hit on the first real deployment.

6 Alert Rules

Alert	Fires when
NodeDown	probe fails for 2 min
NodeSlow	up but > 1 s for 5 min
ModuleFullyDown	every node of a module in one env is down

"for:" durations stop one blip from paging anyone.

7 Reading the Signals

curl / probe result	Means
`200`	healthy
`401`	wrong monitoring password
`500`	app reached, but failing
`000`	**no TCP connection** – firewall / routing, not the app

Ping works but port fails = port-specific firewall rule.

8 Results

6	3	5
demo nodes	alert rules	tests

- End-to-end run verified: healthy, slow (1.5 s), HTTP 500 and unreachable nodes all detected; alerts fired.
- `promtool` validates config + rules in CI; CI then boots the stack and waits for every target.

9 Key Concepts Learned

- **Pull model:** Prometheus scrapes on a schedule; blackbox turns "is this URL OK?" into metrics.
- **Labels** (env/module/node) make one query serve every filter and dashboard.
- **Configuration as code:** dashboards, data sources and targets live in Git.

10 Limitations & Future Work

- Availability + latency only, no JVM internals (by design, see ④).
- Next: Alertmanager → email / Teams notifications.
- Next: run on a dedicated host instead of a laptop / shared VM.

11 Interview Talking Points

Q: Everything shows DOWN – where do you look first?
Prometheus *Targets* page: it separates "can't reach/parse" from "Grafana query is wrong".

Q: Why a CSV?
Non-developers can maintain it; the script guards quality.

Portfolio project · fictional data · MIT licence